SpaceX 火箭发射数据分析

本项目分析 SpaceX 猎鹰 9 号发射数据，探索发射成功率，搭建交互式仪表盘，并使用机器学习模型预测回收成功与否。

1．数据导入与数据预览

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import folium
from folium.plugins import MarkerCluster
import plotly.express as px
import dash
from dash import dcc, html, Input, Output
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 加载官方数据集，不用本地csv
url_csv = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/spacex_launch_dash.csv"
df = pd.read_csv(url_csv)
df.rename(columns={"class":"Class"}, inplace=True)
print(df.head())
```

2．探索性可视化：发射站点与飞行次数散点图

```
plt.figure(figsize=(10,6))
sns.catplot(y="Launch Site", x="Flight Number", hue="Class", data=df, height=6, aspect=1.2)
plt.ylabel("Launch Site")
plt.xlabel("Flight Number")
plt.show()
```

```
<Figure size 1000x600 with 0 Axes>
```

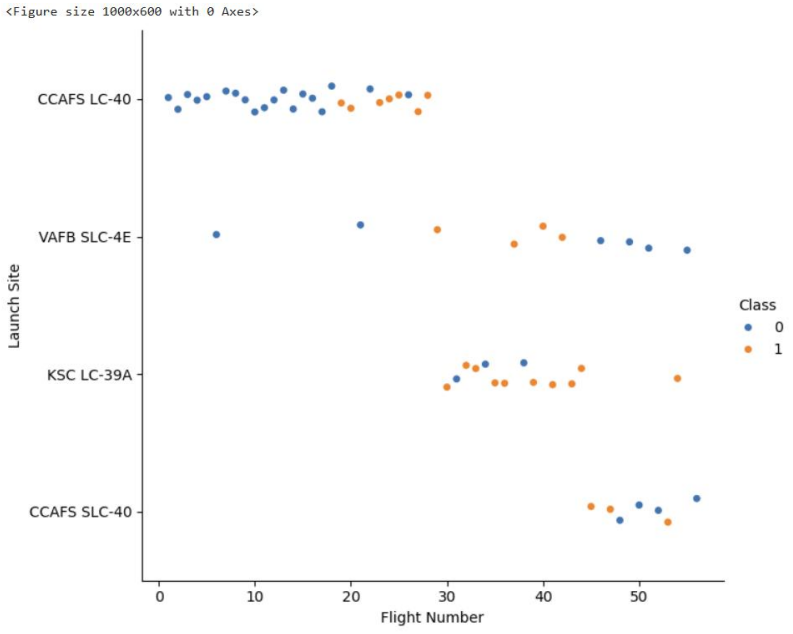


图 1：发射站点与飞行次数分布图

3．发射站点地理分布地图（Folium）

```
spacex_map = folium.Map(location=[28.4, -80.6], zoom_start=5)
cluster = MarkerCluster().add_to(spacex_map)

launch_sites = [
    {"name":"KSC LC-39A","lat":28.6080,"lon":-80.6040},
    {"name":"CCAFS LC-40","lat":28.5619,"lon":-80.5774},
    {"name":"VAFB SLC-4E","lat":34.6320,"lon":-120.6108},
    {"name":"CCAFS SLC-40","lat":28.5632,"lon":-80.5768}
]

for site in launch_sites:
    folium.Marker([site["lat"], site["lon"]], popup=site["name"]).add_to(cluster)

spacex_map
```

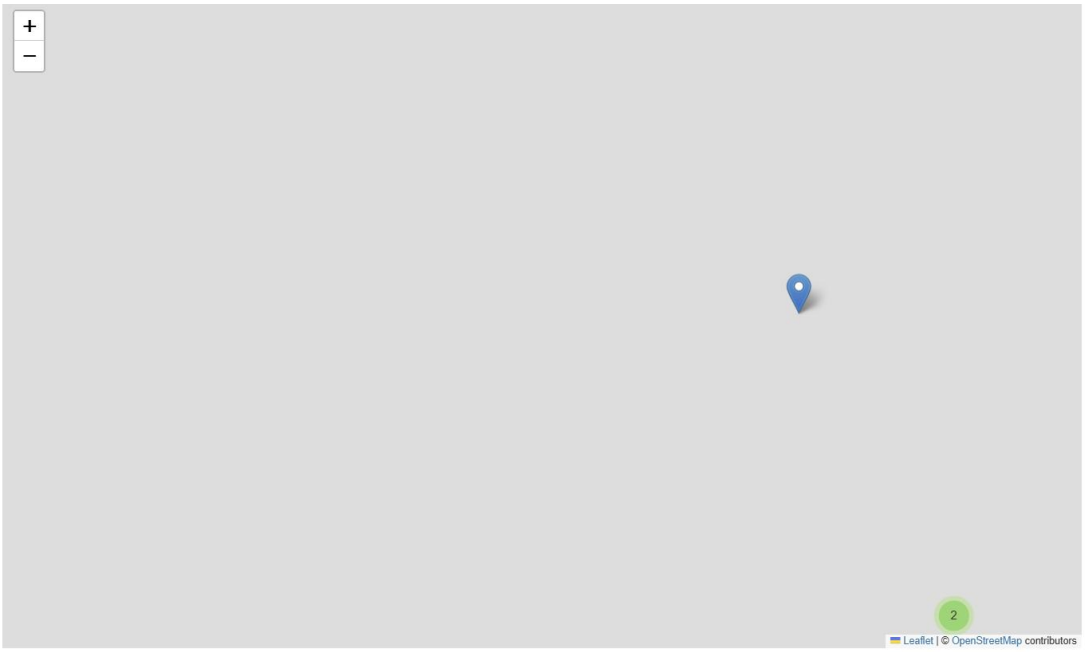


图 2：发射站点地理分布图

4．交互式可视化仪表盘（Dash）

```python
app = dash.Dash(__name__)

app.layout = html.Div([
    html.H1("SpaceX Launch Records Dashboard", style={"textAlign":"center"}),
    dcc.Dropdown(
        id='site-dropdown',
        options=[
            {'label':'All Sites','value':'ALL'},
            {'label':'KSC LC-39A','value':'KSC LC-39A'},
            {'label':'CCAFS LC-40','value':'CCAFS LC-40'},
            {'label':'VAFB SLC-4E','value':'VAFB SLC-4E'},
            {'label':'CCAFS SLC-40','value':'CCAFS SLC-40'}
        ],
        value='ALL',
        placeholder="Select a Launch Site here",
        searchable=True
    ),
    html.Br(),
    dcc.Graph(id='success-pie-chart'),
    html.Br(),
    dcc.RangeSlider(id='payload-slider', min=0, max=10000, step=1000, value=[0,10000]),
    dcc.Graph(id='success-payload-scatter-chart'),
])

@app.callback(
    Output(component_id='success-pie-chart', component_property='figure'),
    Input(component_id='site-dropdown', component_property='value')
)
def get_pie_chart(entered_site):
    filtered_df = df
    if entered_site == 'ALL':
        fig = px.pie(filtered_df, values='Class', names='Launch Site', title='Total Success Launches By Site')
    else:
        filtered_df = df[df['Launch Site'] == entered_site]
        fig = px.pie(filtered_df, names='Class', title=f'Success vs Failure for {entered_site}')
    return fig

@app.callback(
    Output(component_id='success-payload-scatter-chart', component_property='figure'),
    [Input(component_id='site-dropdown', component_property='value'),
     Input(component_id='payload-slider', component_property='value')]
)
def get_scatter_plot(entered_site, payload):
    low, high = payload
    mask = (df['Payload Mass (kg)'] > low) & (df['Payload Mass (kg)'] < high)
    filtered_df = df[mask]
    if entered_site != 'ALL':
        filtered_df = filtered_df[filtered_df['Launch Site'] == entered_site]
    fig = px.scatter(filtered_df, x="Payload Mass (kg)", y="Class", color="Booster Version Category")
    return fig

if __name__ == '__main__':
    app.run(debug=True)
```

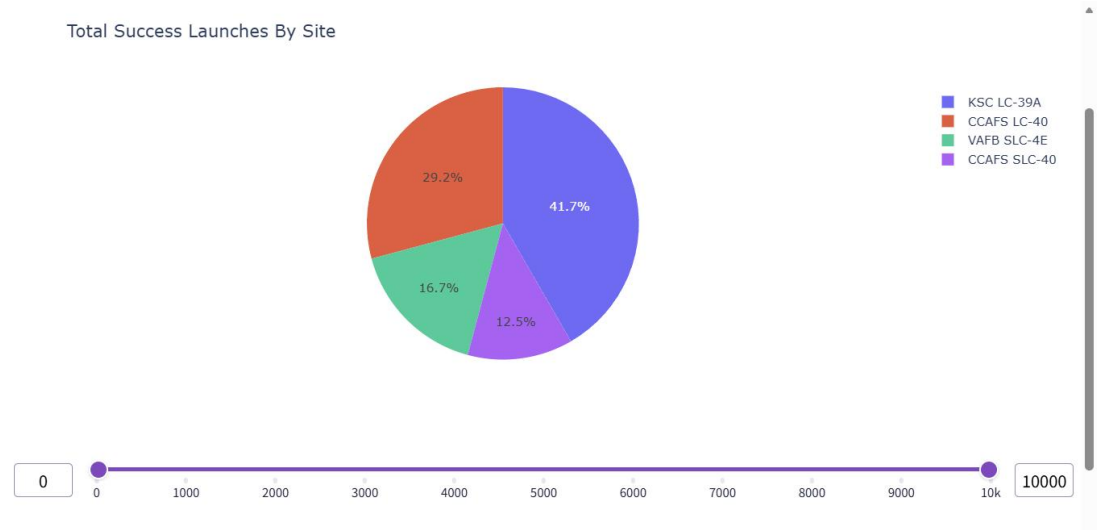




图 3：交互式 Dash 可视化仪表盘

5．机器学习预测模型与结果

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 挑选特征
features = df[['Flight Number','Payload Mass (kg)']]
y = df['Class']

scaler = StandardScaler()
X = scaler.fit_transform(features)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# SVM模型
svm = SVC()
svm.fit(X_train, y_train)
y_pred_svm = svm.predict(X_test)
print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))

# 决策树模型
tree = DecisionTreeClassifier()
tree.fit(X_train, y_train)
y_pred_tree = tree.predict(X_test)
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_tree))
```

```
SVM Accuracy: 0.9166666666666666
Decision Tree Accuracy: 0.8333333333333334
```

图 4：模型准确率输出结果